

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1412 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization*

(BL4) Assessment Tool Weightage: 10%

Duration :60 Mins

QUESTIONS: 10 Total Marks:50 Annexure A

DATA INTERPRETATION

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1/8/26

CO4-AT3-DATA INTERPRETATION

1. 3-Address Code & Expression Trees:

$$t_1 = a * b$$

$$t_2 = t_1 + c$$

$$t_3 = t_2 - d$$

i) Analyze the sequence of computations:

First Computation:

$$t_1 = a * b$$

Second Computation:

$$t_2 = t_1 + c$$

Third Computation:

$$t_3 = t_2 - d$$

The equivalent arithmetic expression is:

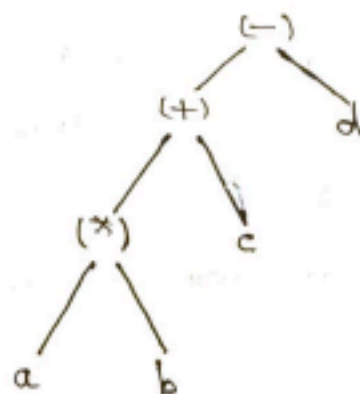
$$(a * b) + c - d$$

The computation follows the dependency

$$a, b \rightarrow t_1 \rightarrow t_2 \rightarrow t_3$$

ii) Construct the equivalent expression tree:

$$((a * b) + c) - d$$



iii) Role of temporary variables:

- * Store intermediate results
- * Avoid unnecessary recomputation
- * Improve execution efficiency
- * Simplify Code generation

2. Symbol Table and Memory Allocation:

Variable	Type	Size (bytes)
p	int	4
q	float	8
r	char	1
s	double	8

i) Memory Allocation:

- p - int : allocates 4 bytes of memory.
q - float : Allocates 8 bytes of memory according to the given symbol table.
r - char : allocates 1 byte of memory.
s - double : allocates 8 bytes of memory.

ii)

$$\begin{aligned}\text{Total Memory} &= 4 + 8 + 1 + 8 \\ &= 21 \text{ bytes}\end{aligned}$$

iii) Alignment & Padding:

Alignment places variables at suitable memory addresses.

Padding may add extra unused bytes between variables to satisfy alignment requirements.

Hence, actual storage may be greater than 21 bytes.

3. Code Optimization & Common Subexpression Elimination

$$t_1 = x + y$$

$$t_2 = t_1 * z$$

$$t_3 = x + y$$

$$t_4 = t_3 - w$$

(i) First Operation

$$t_1 = x + y$$

Second Operation:

$$t_2 = t_1 * z$$

Third Operation:

$$t_3 = x + y$$

Fourth Operation:

$$t_4 = t_3 - w$$

Therefore:

$$t_1 = x + y$$

$$t_2 = (x + y) * z$$

$$t_3 = x + y$$

$$t_4 = (x + y) - w$$

ii) Common Subexpression:

$$x + y$$

It occurs twice:

$$t_1 = x + y$$

$$t_3 = x + y$$

iii) Optimized Three-Address Code:

$$t_1 = x + y$$

$$t_2 = t_1 * z$$

$$t_4 = t_1 - w$$

The statement

$$t_3 = x + y$$

t_1 directly used in

$$t_4 = t_1 - w$$

Before Optimization:

$$x + y \rightarrow t_1$$



used for t_2

$$x + y \rightarrow t_3$$



used for t_4

After Optimization:

$$x + y \rightarrow t_1$$



4. Boolean Expressions, Short-Circuit Evaluation & Backpatching:

if $(a < b \parallel c > d)$

Intermediate Code:

if $a < b$ goto L_1

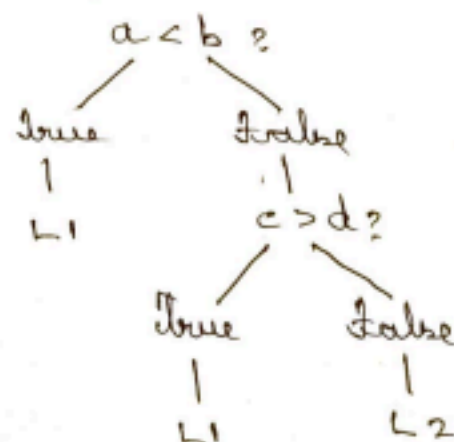
if $c > d$ goto L_1

goto L_2

$L_1: \dots$

$L_2: \dots$

i) Control flow of the intermediate code:



ii) Short-Circuit Evaluation for OR (||) operator:

A || B

$(a < b) || (c > d)$

→ $a < b$ is true, the complete OR expression is true, therefore, $c > d$ is not evaluated & control directly goes to L1.

→ If $a < b$ is false, evaluate $c > d$

→ If $c > d$ is true, the control goes to L1

→ If both conditions are false, control goes

iii) Truelist:

Contains the list of incomplete jumps that eventually go to the true destination.

Falselist:

Contain jumps that should eventually go to the false destination.

Backpatching:

Resolves the pending jump targets.

5. Procedure Calls & Activation Records:

total = compute(x, y)

Intermediate Representation:

param x

param y

call compute, 2

i) Parameter Passing Mechanism:

The intermediate code uses the parameter passing mechanism where the arguments are 1st placed as parameters before calling the procedure.

∴ The function receives x & y as its input parameters & performs the required computation.

ii) Major Components :

1. Parameters
2. Return value
3. Return Address
4. Local variables
5. Control information

Purpose of Activation Record:

An activation record is a block of memory created for a procedure or function. It stores the information required during the execution of that function.

iii) Function Call:

$\text{total} = \text{compute}(x, y)$

The sequence is

param x

param y

call compute, 2.